Software Engineering-2

Task Flow

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete description of the functional and technical requirements for the TaskFlow project management platform. This document acts as a reference for developers, testers, and stakeholders to ensure that the system is implemented according to the required business logic and microservices architecture.

1.2 Scope

TaskFlow is a microservices-based task and project management platform designed to help teams collaborate efficiently.

Users: The platform allows users to register, log in securely, create projects, manage tasks, upload attachments, and communicate through comments.

Project Managers/Admins: Users with administrative privileges can manage project members, assign tasks, review system statistics, and control permissions.

The system is developed using Spring Boot for backend microservices, API Gateway for routing and security, MySQL for data persistence, Kafka for notifications/events, and Docker Compose for containerized deployment.

1.3 Definitions, Acronyms, and Abbreviations

JWT (JSON Web Token): A secure token used for stateless authentication.

Microservices: An architecture where the application is divided into small independent services.

API Gateway: A centralized entry point that routes requests and validates security.

Kafka: A distributed event-streaming platform used for notifications and asynchronous communication.

RBAC (Role-Based Access Control): A mechanism for controlling access based on user roles and permissions.

SRS: Software Requirements Specification document.

2. Overall Description

2.1 System Perspective

The system consists of multiple independent microservices:

- Auth Service

- Project Service

- Task Service

- Notification Service

- API Gateway

Services communicate through REST APIs and Kafka events. Security is centralized at the API Gateway using JWT authentication.

2.2 User Classes and Characteristics

User: Can register, log in, create/manage projects, create tasks, upload attachments, and add comments.

Project Manager/Admin: Can manage project members, assign tasks, manage permissions, and monitor project statistics.

2.3 Design and Implementation Constraints

Database Isolation: Each service manages its own database logic independently.

Statelessness: Authentication relies entirely on JWT tokens.

Environment: Development uses Java 21, Spring Boot 3.x, Docker Compose, Kafka, and MySQL.

Scalability: Each microservice is independently deployable and scalable.

3. Functional Requirements

3.1 Authentication & Authorization

FR-01: The system shall allow users to register and log in securely.
FR-02: The system shall generate JWT tokens after successful authentication.
FR-03: The system shall support RBAC using roles and permissions.
FR-04: The system shall allow admins to approve pending users.
FR-05: The system shall allow users to retrieve their profile information.
FR-06: The system shall support permission management for roles.

3.2 Project Management

FR-07: Users shall be able to create, update, delete, and view projects.
FR-08: Users shall be able to add or remove project members.
FR-09: Users shall be able to view their assigned projects.
FR-10: The system shall provide dashboard statistics for projects.

3.3 Task Management

FR-11: Users shall be able to create, update, delete, and view tasks.
FR-12: Users shall be able to assign tasks to team members.
FR-13: Users shall be able to upload task attachments.
FR-14: Users shall be able to create and manage comments on tasks.
FR-15: Users shall be able to retrieve tasks by project or by assigned user.

3.4 Notifications

FR-16: The system shall generate notifications for important events.
FR-17: Users shall be able to retrieve unread notifications.
FR-18: Users shall be able to delete notifications.FR-19: The system shall support asynchronous notification processing using Kafka.

4. Non-Functional Requirements

4.1 Security

The system shall use BCrypt for password hashing.

All API requests shall be protected using JWT authentication.

HTTPS shall be used in production deployments.

4.2 Performance

Task retrieval and project operations shall respond within acceptable response times (less than 2 seconds under normal load).

4.3 Scalability & Availability

Each service shall run in separate Docker containers.

The system shall support independent scaling of services.

Kafka and Docker health checks shall ensure reliability.

4.4 Maintainability

The system shall use Swagger/OpenAPI documentation.

The system shall follow layered architecture and clean code practices.

Logging and exception handling shall be centralized.

5.Design Patterns Used in the System

The system was developed using several software design patterns and architectural approaches to improve maintainability, scalability, and code organization. The following sections explain the most important patterns identified in the project structure and implementation.

1. Microservices Architecture

The project is divided into multiple independent services such as authentication, task management, notifications, project management, and API gateway services. Each service handles a specific responsibility, which makes the system easier to maintain and scale.

2. API Gateway Pattern

The API Gateway acts as a single entry point for client requests. It is responsible for request routing, authentication handling, and communication between services.

3. Repository Pattern

Repository classes are used to separate database operations from business logic. This improves code readability and simplifies data access management.

4. Service Layer Pattern

The business logic of the application is implemented inside service classes. This layer works between controllers and repositories to organize application functionality clearly.

5. DTO Pattern

Data Transfer Objects (DTOs) are used to transfer data between different layers of the application. This approach helps reduce direct dependency on entity classes and improves security.

6. Dependency Injection

Spring Framework provides dependency injection to manage object creation automatically. This reduces tight coupling between classes and improves flexibility.

7. Singleton Pattern

Most Spring Beans are managed as singleton objects by default. This allows shared use of service and repository instances across the application.

8. Factory Pattern

The Spring IoC container creates and manages application objects automatically, which reflects the concept of the Factory Pattern.

9. Filter Pattern

Filters are used to process requests before reaching controllers. For example, JWT authentication filters validate tokens and user permissions.

10. Strategy Pattern

Spring Security applies different authentication and authorization strategies internally. This allows flexible security configurations within the system.

11. MVC Pattern

The project follows the MVC architecture by separating controllers, services, repositories, and entities into organized layers.

Conclusion

In conclusion, the project applies several well-known software engineering patterns and principles. These patterns help improve scalability, maintainability, readability, and overall system organization. Using Spring Boot and Microservices architecture also provides flexibility for future development and expansion.

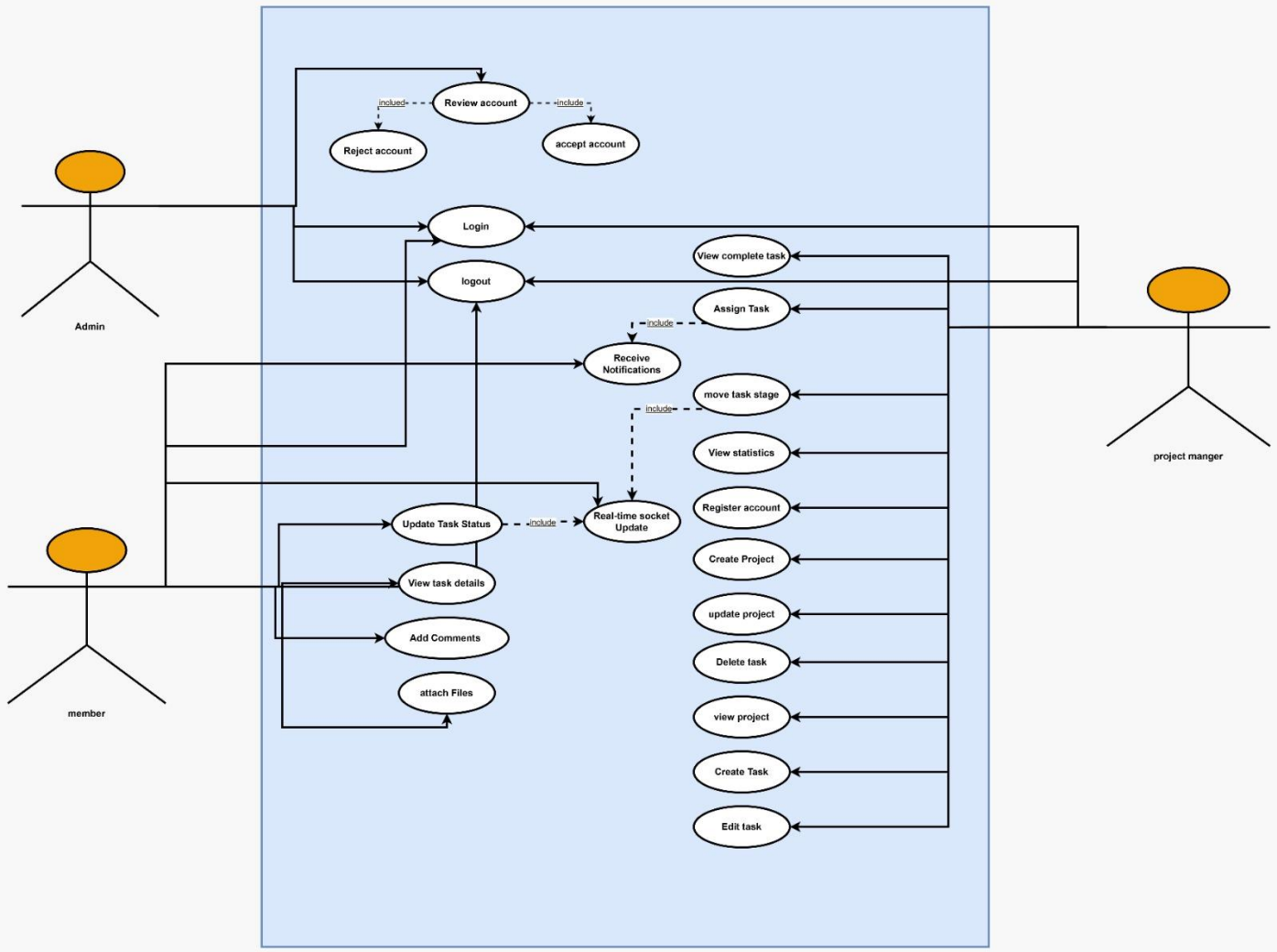
Diagrams

1. Use Case Diagram

The Use Case Diagram illustrates the interaction between the three main actors in the TaskFlow system: Admin, Project Manager, and Member.

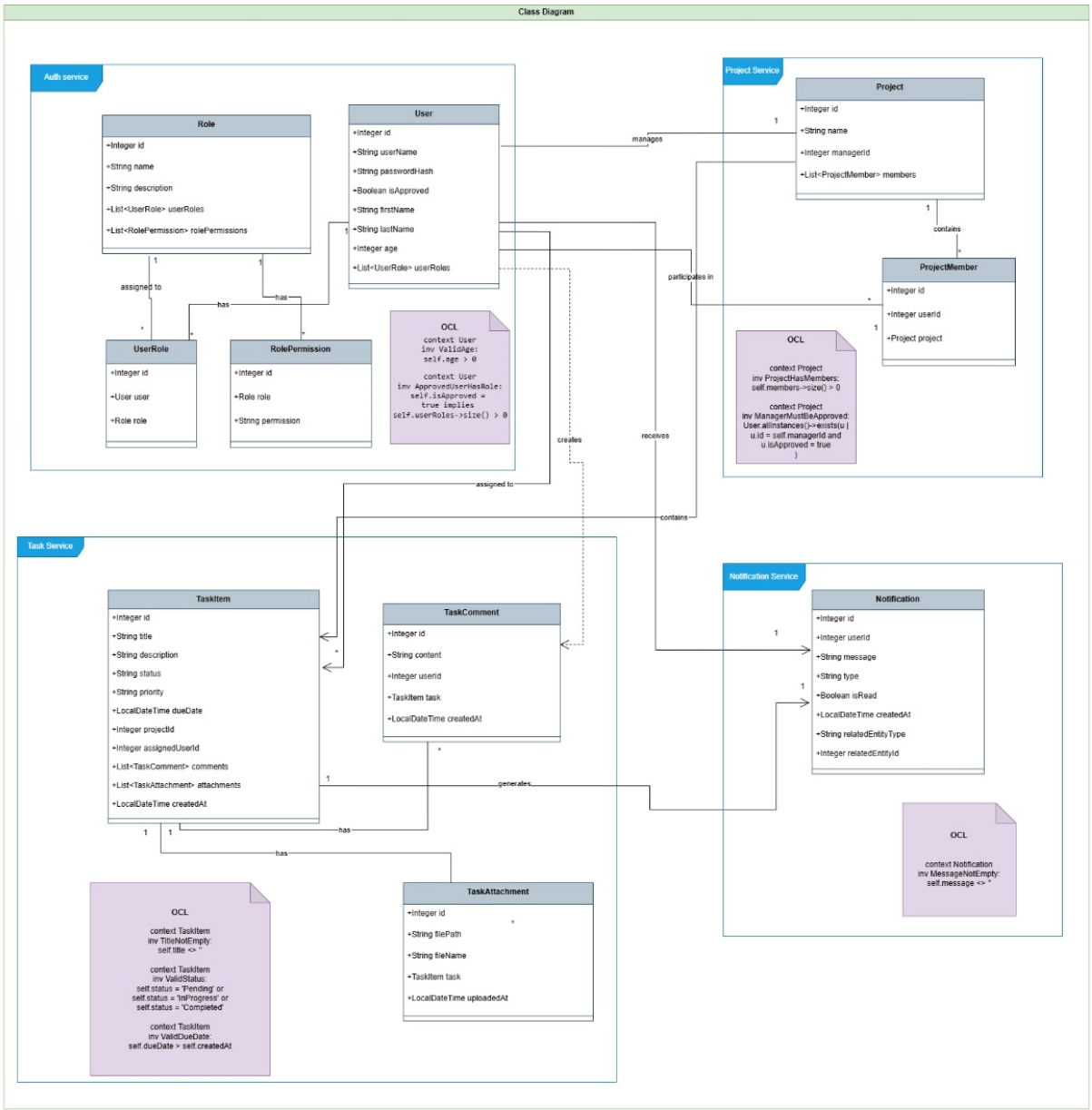
* The Admin manages user accounts by reviewing, accepting, or rejecting registrations, in addition to login/logout operations.
* The Project Manager is responsible for managing projects and tasks, including creating projects, assigning tasks, editing tasks, moving task stages, viewing statistics, and receiving notifications.
* The Member interacts with assigned tasks by viewing task details, updating task status, adding comments, attaching files, and receiving notifications.

The system also supports real-time socket updates and notifications to ensure instant communication and synchronization between users.



Diagrams

2-Class Diagram (All System)



OCL For Class Diagram (All System)

-- User Constraints

context User
inv UniqueUsername:
User.allInstances()->isUnique(userName)
context User
inv ValidAge:
age > 0
context User
inv ApprovedUserHasRole:
isApproved = true implies userRoles->size() > 0

- Role Constraints
context Role
inv UniqueRoleName:
Role.allInstances()->isUnique(name)
context Role
inv RoleMustHavePermission:
rolePermissions->size() > 0

-- Project Constraints
context Project
inv ProjectMustHaveManager:
managerId > 0
context Project
inv ProjectNameNotEmpty:
name <> ''
context Project
inv ProjectMustContainMembers:
members->size() > 0

-- TaskItem Constraints
context TaskItem
inv TaskTitleNotEmpty:
title <> ''
context TaskItem
inv ValidTaskStatus:
status = 'TODO' or
status = 'IN_PROGRESS' or

status = 'DONE'
context TaskItem
inv ValidPriority:
priority = 'LOW' or
priority = 'MEDIUM' or
priority = 'HIGH'
context TaskItem
inv DueDateAfterCreation:
dueDate > createdAt
context TaskItem
inv AssignedUserRequired:
assignedUserId > 0

-- Task Comment Constraints
context TaskComment
inv CommentContentNotEmpty:
content <> ''
context TaskComment
inv CommentMustBelongToTask:
task <> null

-- Task Attachment Constraints
context TaskAttachment
inv FileNameNotEmpty:
fileName <> ''
context TaskAttachment
inv FilePathNotEmpty:
filePath <> ''
context TaskAttachment
inv AttachmentMustBelongToTask:
task <> null
-- Notification Constraints
context Notification
inv NotificationMessageNotEmpty:
message <> ''
context Notification
inv RelatedEntityExists:
relatedEntityId > 0

context Notification
inv NotificationTypeNotEmpty:
type <> ''

-- Project Member Constraints
context ProjectMember
inv UserMustExist:
userId > 0
context ProjectMember
inv ProjectMustExist:
project <> null

-- RolePermission Constraints
context RolePermission
inv PermissionNotEmpty:
permission <> ''
context RolePermission
inv RoleMustExist:
role <> null

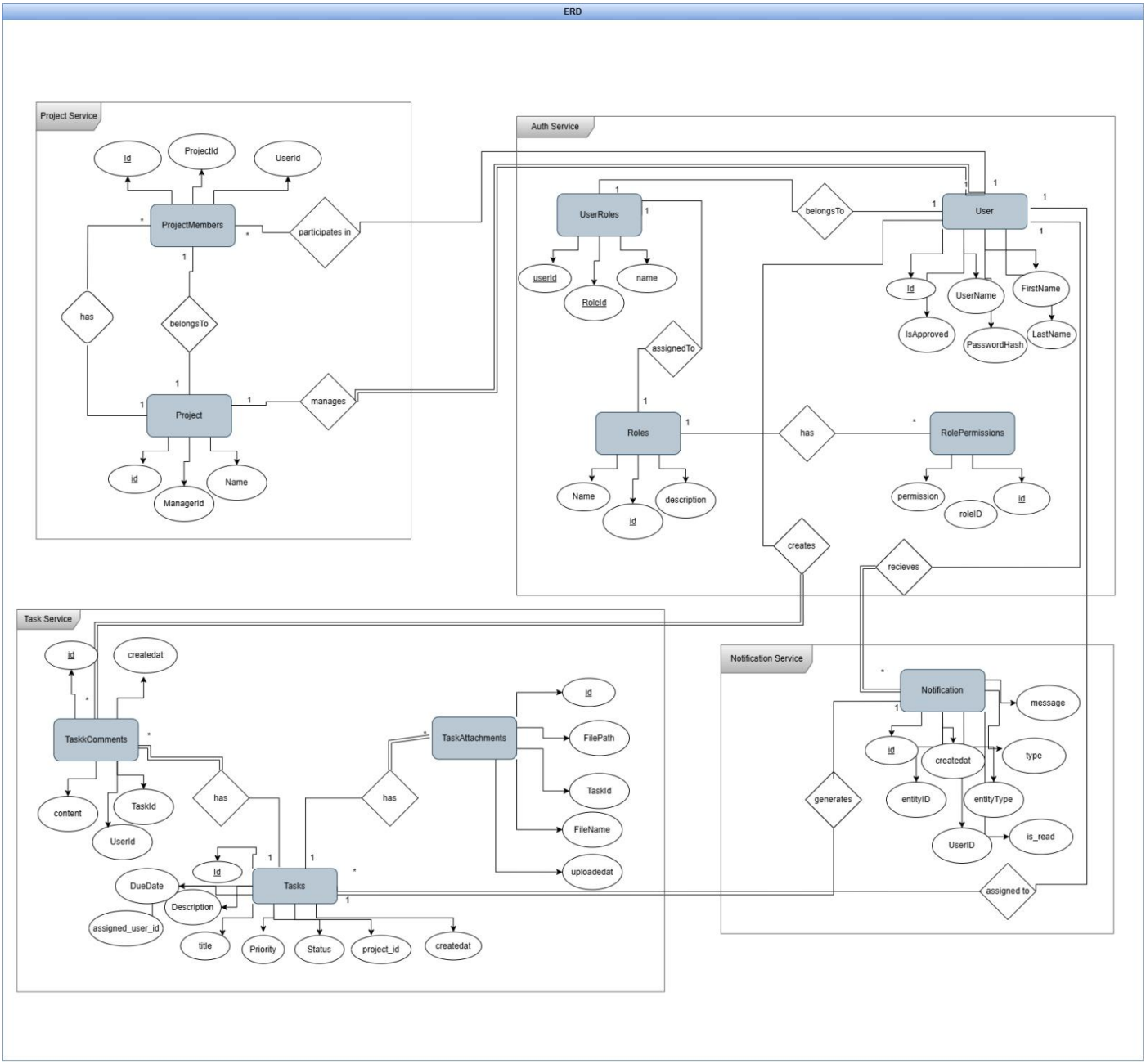
-- UserRole Constraints

context UserRole
inv UserRoleMustHaveUser:
user <> null

context UserRole
inv UserRoleMustHaveRole:
role <> null

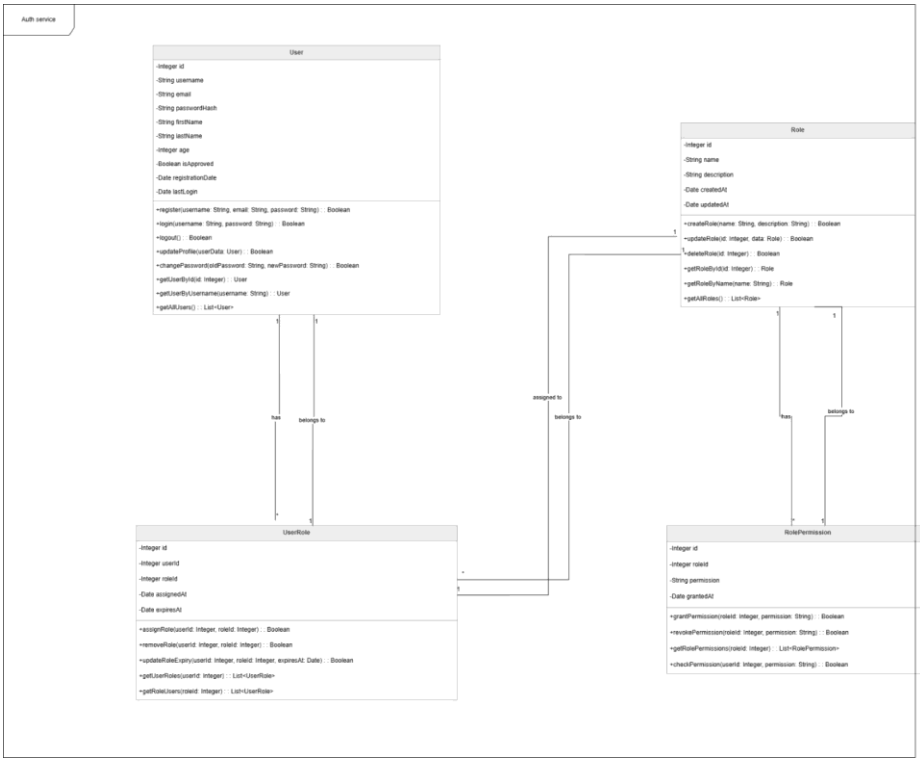
Diagrams

3. ERD (All System)



Diagrams

4-Class Diagram (Auth service)



OCL For Class Diagram

-- User Constraints

context User
inv UniqueUsername:
  User.allInstances()->isUnique(username)
context User
inv UniqueEmail:
  User.allInstances()->isUnique(email)
context User
inv ValidAge:
  age >= 18

context User
inv PasswordNotEmpty:
  passwordHash <> ''
context User
inv ApprovedUsersCanLogin:
  isApproved = true implies lastLogin <> null
context User::register(username : String, email : String, password : String) : Boolean
pre ValidInput:
  username <> '' and email <> '' and password <> ''
post UserCreated:
  result = true implies
  User.allInstances()->exists(u |
    u.username = username and
    u.email = email
)

-- Role Constraints

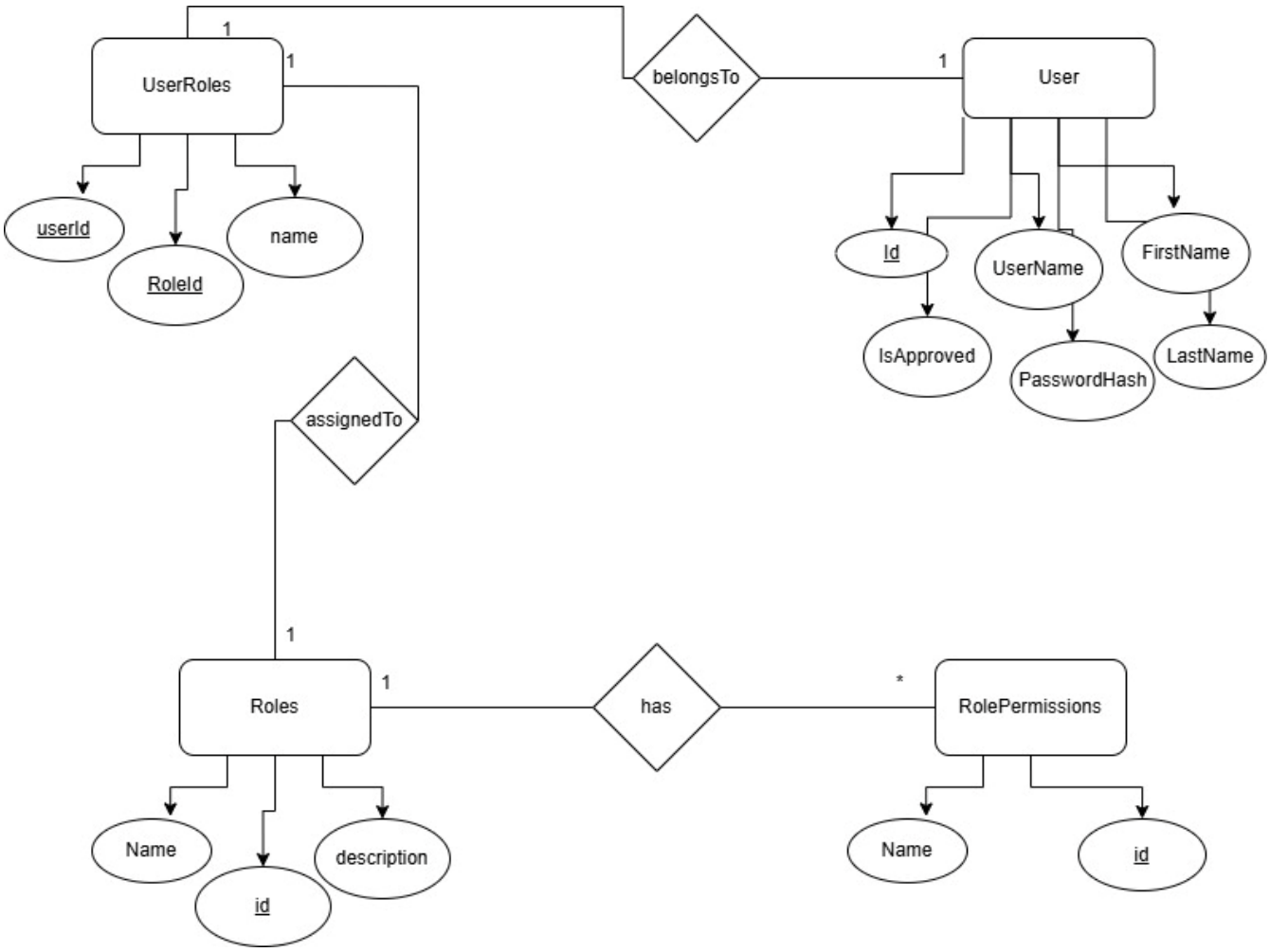
context Role
inv UniqueRoleName:
  Role.allInstances()->isUnique(name)
context Role
inv DescriptionRequired:
  description <> ''
context Role::createRole(name : String, description : String) : Boolean

```
pre ValidRoleData:
  name <> '' and description <> ''
post RoleAdded:
  result = true implies
  Role.allInstances()->exists(r | r.name = name)
-- UserRole Constraints
context UserRole
inv ValidAssignment:
  userId > 0 and roleId > 0
context UserRole
inv ExpiryAfterAssignment:
  expiresAt <> null implies expiresAt > assignedAt
context UserRole
inv UniqueUserRole:
  UserRole.allInstances()->forAll(ur1, ur2 |
    ur1 <> ur2 implies
    not (ur1.userId = ur2.userId and ur1.roleId = ur2.roleId)
  )
context UserRole::assignRole(userId : Integer, roleId : Integer) : Boolean
pre UserAndRoleExist:
  User.allInstances()->exists(u | u.id = userId) and
  Role.allInstances()->exists(r | r.id = roleId)
post RoleAssigned:
  result = true implies
  UserRole.allInstances()->exists(ur |
    ur.userId = userId and ur.roleId = roleId
  )
-- RolePermission Constraints
context RolePermission
inv PermissionNotEmpty:
  permission <> ''
context RolePermission
inv ValidRole:
  roleId > 0
context RolePermission
inv UniquePermissionPerRole:
  RolePermission.allInstances()->forAll(rp1, rp2 |
    rp1 <> rp2 implies
    not (rp1.roleId = rp2.roleId and rp1.permission = rp2.permission)
  )
context RolePermission::grantPermission(roleId : Integer, permission : String) : Boolean
pre ValidPermission:
  permission <> ''
post PermissionGranted:
  result = true implies
  RolePermission.allInstances()->exists(rp |
    rp.roleId = roleId and rp.permission = permission
  )
context RolePermission::revokePermission(roleId : Integer, permission : String) : Boolean
pre PermissionExists:
  RolePermission.allInstances()->exists(rp |
    rp.roleId = roleId and rp.permission = permission
  )

post PermissionRemoved:
  result = true implies
  not RolePermission.allInstances()->exists(rp |
    rp.roleId = roleId and rp.permission = permission
   )
```

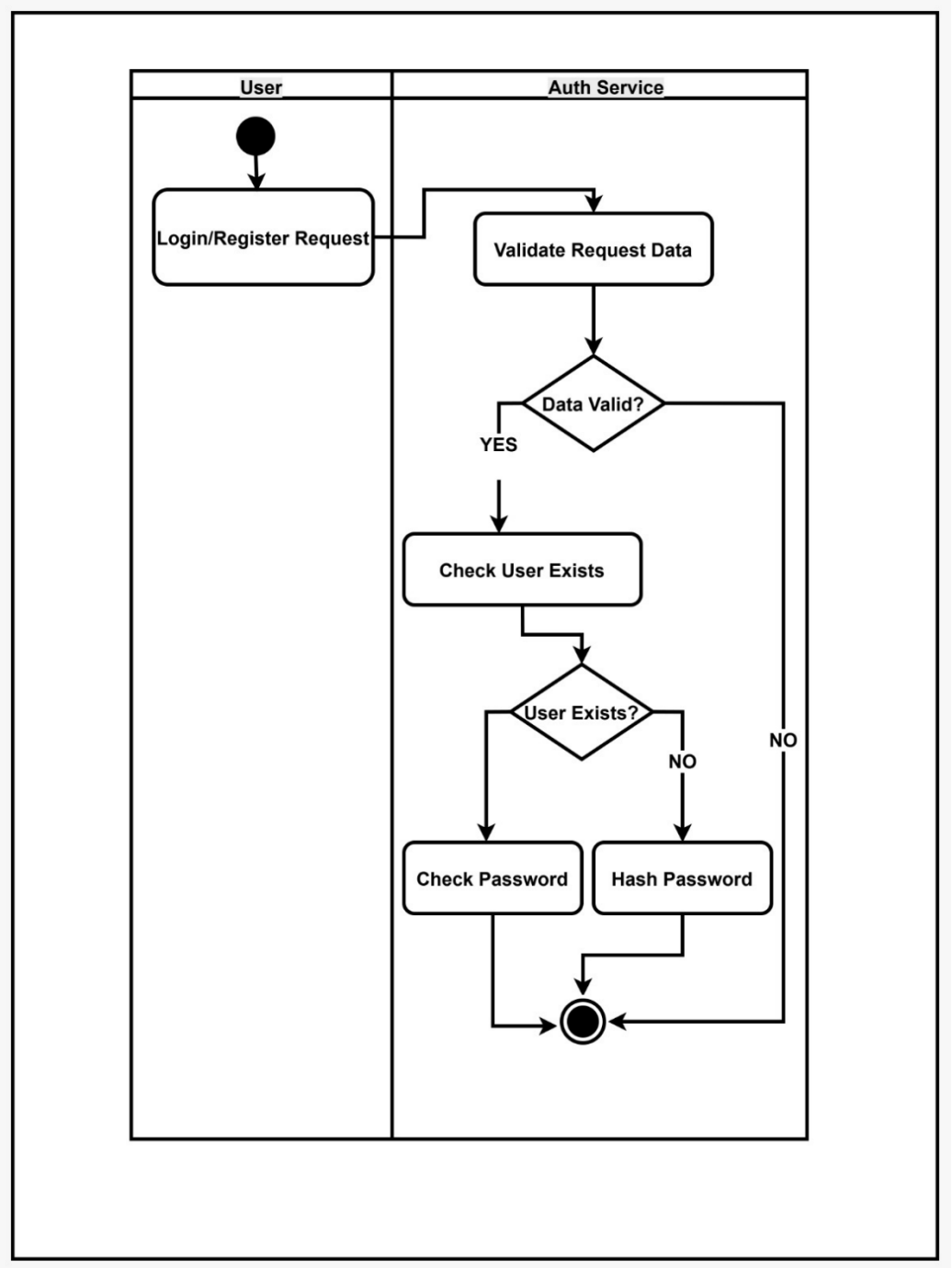
Diagrams

5-ERD (Auth service)



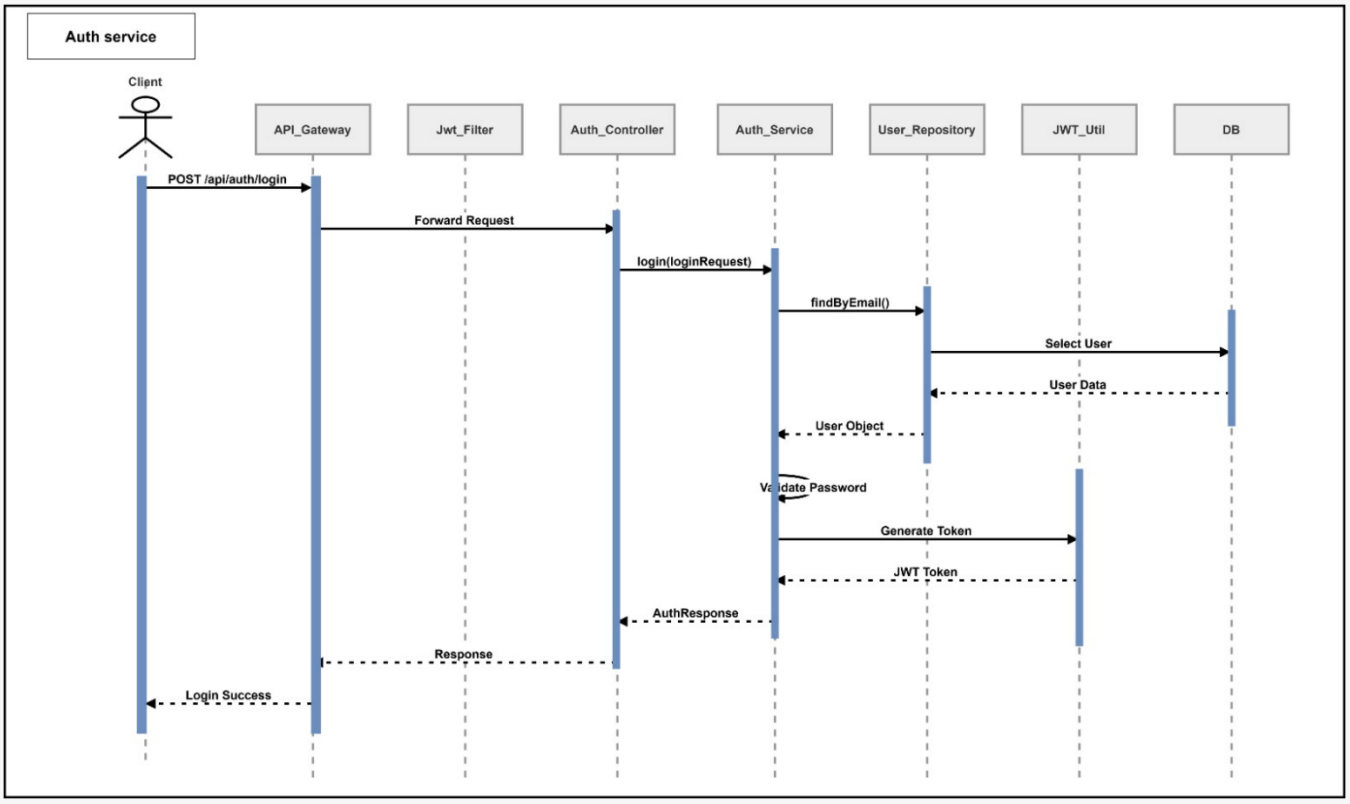
Diagrams

6-Activity Diagram (Auth service)

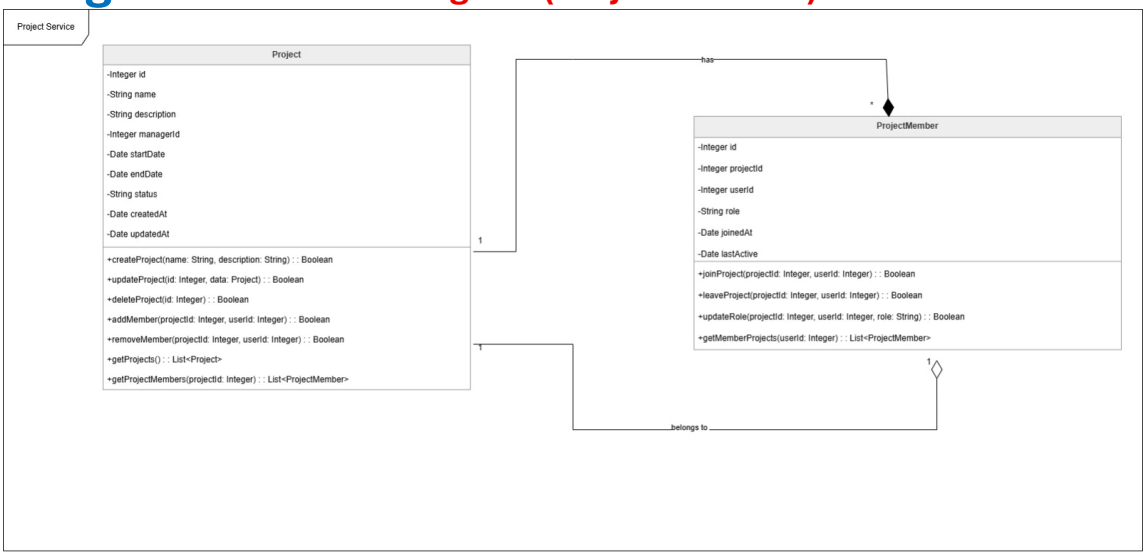


Diagrams

7-Sequence Diagram (Auth service)



Diagrams 8-Class Diagram (Project service)



OCL For Class Diagram

-- Project Class Constraints

context Project
inv ValidProjectName:
  self.name <> '' and self.name.size() > 2

context Project
inv ValidDates:
  self.startDate < self.endDate

context Project
inv ValidStatus:
  self.status = 'Planned' or
  self.status = 'In Progress' or
  self.status = 'Completed' or
  self.status = 'Cancelled'

context Project
inv ManagerExists:
  self.managerId > 0

context Project
inv CreatedBeforeUpdated:
  self.createdAt <= self.updatedAt

context Project
inv UniqueMembers:
  self.projectMember->isUnique(userId)

-- Project Operations

context Project::createProject(
  name : String,
  description : String
) : Boolean
pre ValidInput:
  name <> '' and description <> ''
post ProjectCreated:
  result = true implies
  Project.allInstances()->exists(p |
    p.name = name and
    p.description = description
)

context Project::updateProject(
  id : Integer,
  data : Project
) : Boolean
pre ProjectExists:

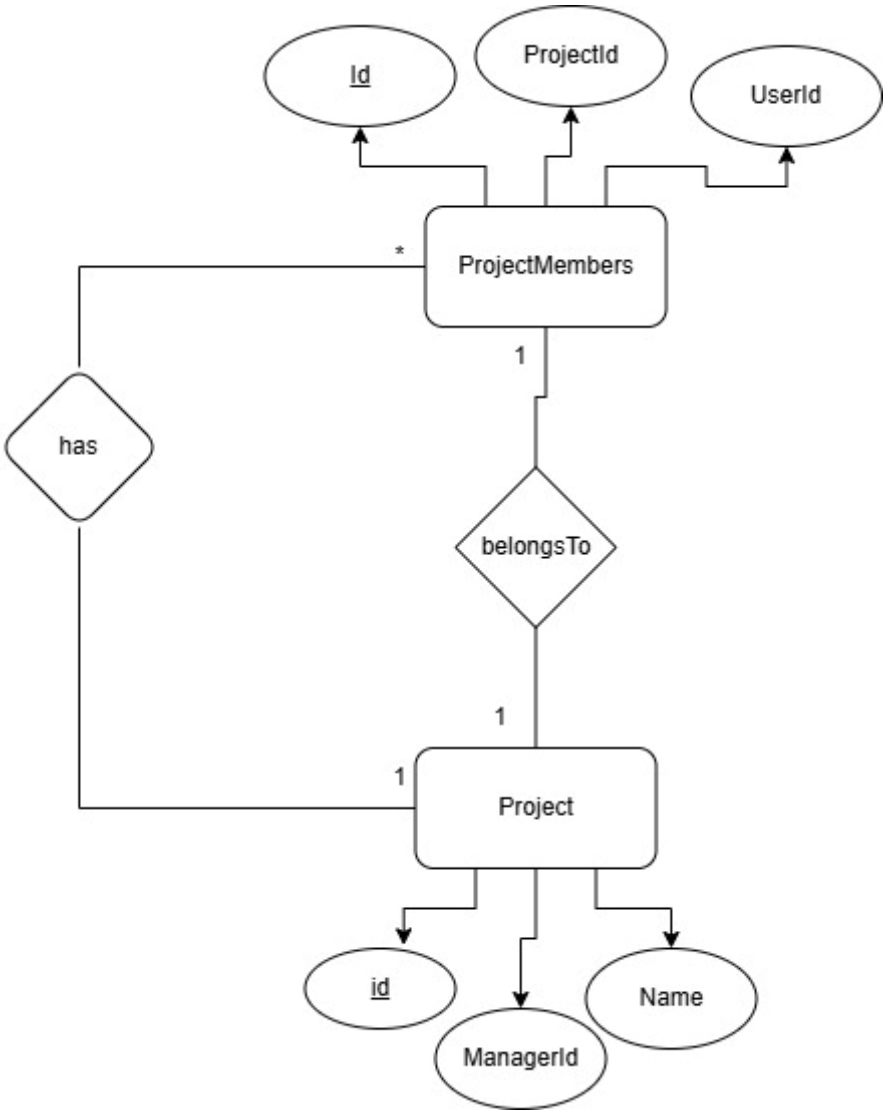
```
  Project.allInstances()->exists(p | p.id = id)
post ProjectUpdated:
  result = true implies
  Project.allInstances()->exists(p |
    p.id = id and
    p.name = data.name
  )
context Project::deleteProject(
  id : Integer
) : Boolean
pre ProjectExists:
  Project.allInstances()->exists(p | p.id = id)
post ProjectDeleted:
  result = true implies
  Project.allInstances()->forAll(p | p.id <> id)
context Project::addMember(
  projectId : Integer,
  userId : Integer
) : Boolean
pre ProjectExists:
  Project.allInstances()->exists(p | p.id = projectId)
pre MemberNotAlreadyExists:
  ProjectMember.allInstances()->forAll(pm |
    not (pm.projectId = projectId and pm.userId = userId)
  )
post MemberAdded:
  result = true implies
  ProjectMember.allInstances()->exists(pm |
    pm.projectId = projectId and
    pm.userId = userId
  )
context Project::removeMember(
  projectId : Integer,
  userId : Integer
) : Boolean
pre MemberExists:
  ProjectMember.allInstances()->exists(pm |
    pm.projectId = projectId and
    pm.userId = userId
  )
post MemberRemoved:
  result = true implies
  ProjectMember.allInstances()->forAll(pm |
    not (pm.projectId = projectId and pm.userId = userId)
  )
-- ProjectMember Class Constraints
context ProjectMember
inv ValidRole:
  self.role <> ''
context ProjectMember
inv JoinedBeforeLastActive:
  self.joinedAt <= self.lastActive
context ProjectMember
inv ValidUserId:
  self.userId > 0
context ProjectMember
inv ValidProjectId:
  self.projectId > 0
-- ProjectMember Operations
context ProjectMember::joinProject(
```

```
  projectId : Integer,
  userId : Integer
) : Boolean
pre NotJoinedBefore:
  ProjectMember.allInstances()->forAll(pm |
    not (pm.projectId = projectId and pm.userId = userId)
  )
post JoinedSuccessfully:
  result = true implies
  ProjectMember.allInstances()->exists(pm |
    pm.projectId = projectId and
    pm.userId = userId
  )
context ProjectMember::leaveProject(
  projectId : Integer,
  userId : Integer
) : Boolean

pre MemberExists:
  ProjectMember.allInstances()->exists(pm |
    pm.projectId = projectId and
    pm.userId = userId
  )
post LeftSuccessfully:
  result = true implies
  ProjectMember.allInstances()->forAll(pm |
    not (pm.projectId = projectId and pm.userId = userId)
  )
context ProjectMember::updateRole(
  projectId : Integer,
  userId : Integer,
  role : String
) : Boolean
pre MemberExists:
  ProjectMember.allInstances()->exists(pm |
    pm.projectId = projectId and pm.userId = userId )
pre ValidRole:
  role <> ''
post RoleUpdated:
  result = true implies
  ProjectMember.allInstances()->exists(pm |
    pm.projectId = projectId and
    pm.userId = userId and
    pm.role = role)
```

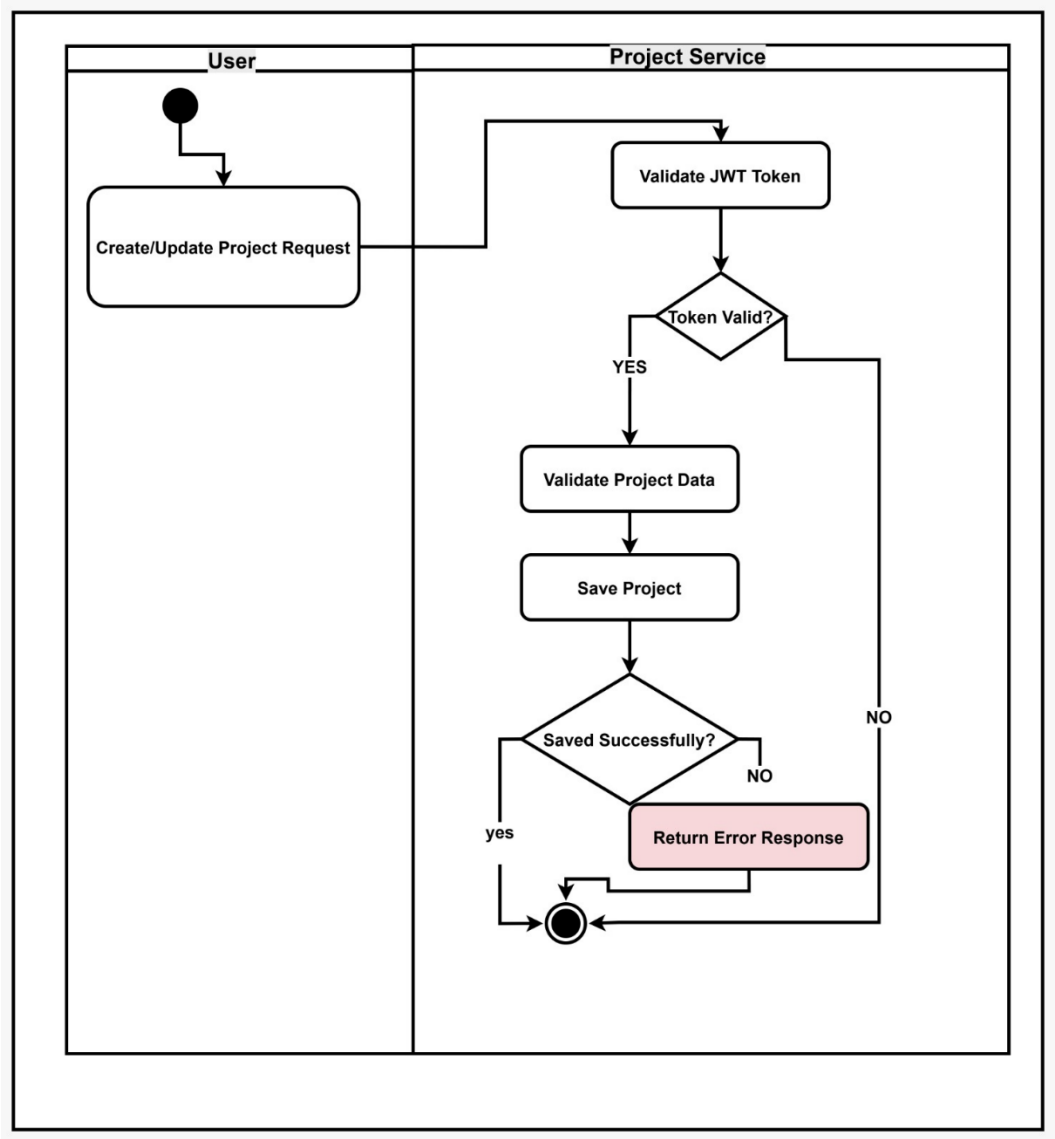
Diagrams

9-ERD (Project service)



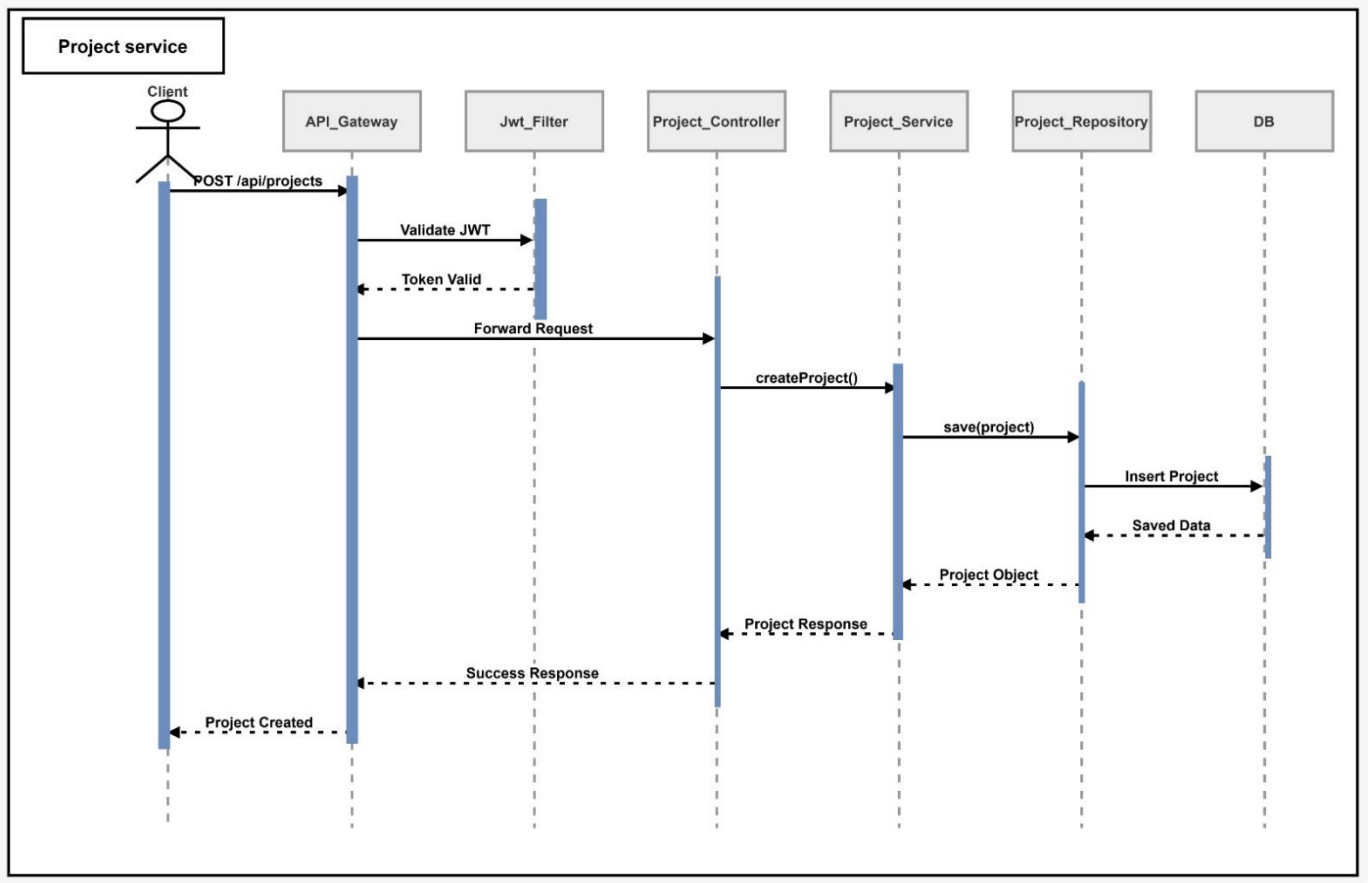
Diagrams

10-Activity Diagram (Project service)



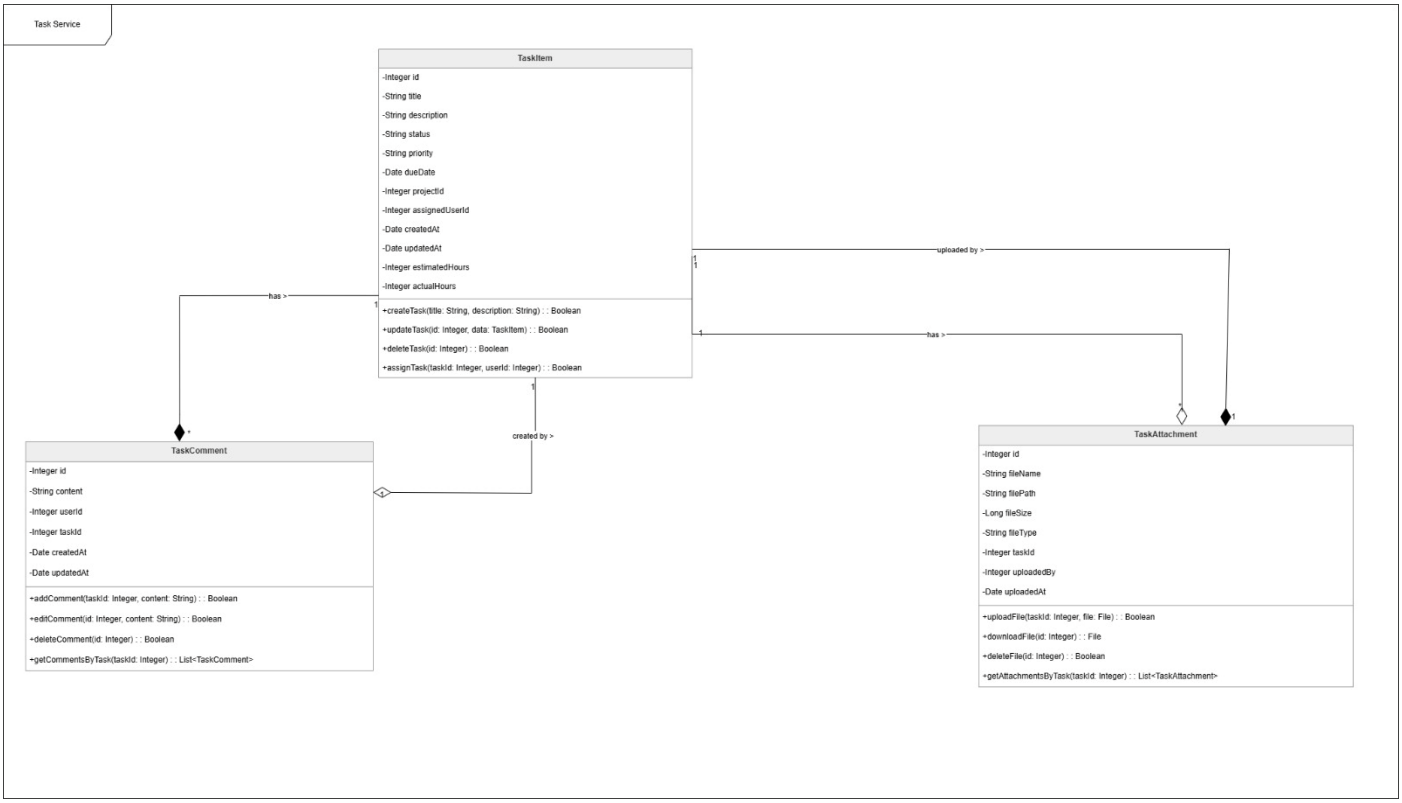
Diagrams

11-Sequence Diagram (Project service)



Diagrams

12-Class Diagram (Task service)



OCL For Class Diagram

-- TaskItem Constraints

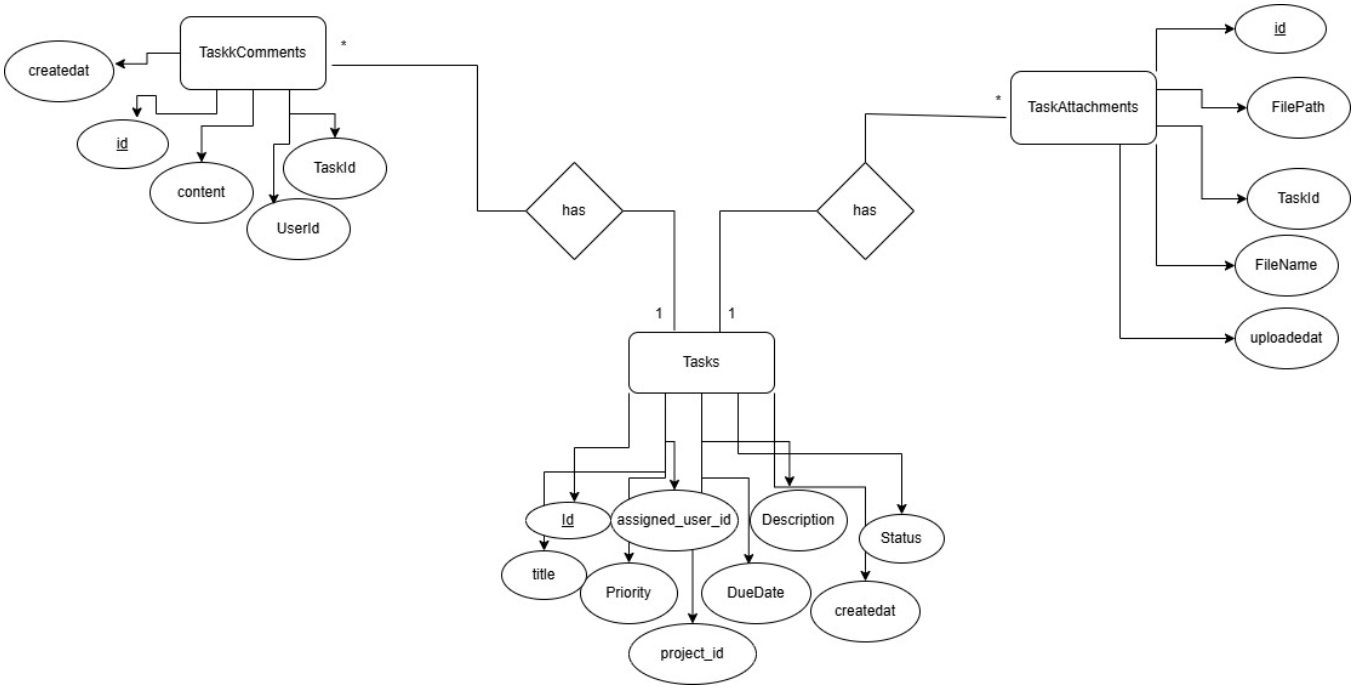
context TaskItem
inv TitleNotEmpty:
  title <> ''
context TaskItem
inv ValidStatus:
  status = 'Pending' or
  status = 'In Progress' or
  status = 'Completed'
context TaskItem
inv ValidPriority:
  priority = 'Low' or
  priority = 'Medium' or
  priority = 'High'
context TaskItem
inv ValidEstimatedHours:
  estimatedHours >= 0
context TaskItem
inv ValidActualHours:
  actualHours >= 0
context TaskItem
inv DueDateAfterCreation:
  dueDate > createdAt
context TaskItem
inv UpdatedAfterCreated:
  updatedAt >= createdAt
context TaskItem::createTask(title : String, description : String) : Boolean
pre ValidTaskData:
  title <> '' and description <> ''

```
post TaskCreated:
  result = true implies
  TaskItem.allInstances()->exists(t |
    t.title = title and
    t.description = description
  )
context TaskItem::assignTask(taskId : Integer, userId : Integer) : Boolean
pre ValidAssignment:
  taskId > 0 and userId > 0
post UserAssigned:
  result = true implies
  TaskItem.allInstances()->exists(t |
    t.id = taskId and
    t.assignedUserId = userId
  )
-- TaskComment Constraints
context TaskComment
inv ContentNotEmpty:
  content <> ''
context TaskComment
inv ValidUser:
  userId > 0
context TaskComment
inv ValidTask:
  taskId > 0
context TaskComment
inv UpdatedAfterCreated:
  updatedAt >= createdAt
context TaskComment::addComment(taskId : Integer, content : String) : Boolean
pre ValidComment:
  content <> ''
post CommentAdded:
  result = true implies
  TaskComment.allInstances()->exists(c |
    c.taskId = taskId and
    c.content = content
  )
context TaskComment::editComment(id : Integer, content : String) : Boolean
pre CommentExists:
  TaskComment.allInstances()->exists(c | c.id = id)
pre NewContentValid:
  content <> ''
post CommentUpdated:
  result = true implies
  TaskComment.allInstances()->exists(c |
    c.id = id and
    c.content = content
  )
-- TaskAttachment Constraints
context TaskAttachment
inv FileNameNotEmpty:
  fileName <> ''
context TaskAttachment
inv FilePathNotEmpty:
  filePath <> ''
context TaskAttachment
inv ValidFileSize:
  fileSize > 0
context TaskAttachment
inv ValidTaskId:
```

```
  taskId > 0
context TaskAttachment
inv ValidUploader:
  uploadedBy > 0
context TaskAttachment
inv UploadedDateValid:
  uploadedAt >= Date.now()
context TaskAttachment::uploadFile(taskId : Integer, file : File) : Boolean
pre ValidUpload:
  taskId > 0
post FileUploaded:
  result = true implies
  TaskAttachment.allInstances()->exists(a |
    a.taskId = taskId
  )
context TaskAttachment::deleteFile(id : Integer) : Boolean
pre FileExists:
  TaskAttachment.allInstances()->exists(a | a.id = id)
post FileDeleted:
  result = true implies
  not TaskAttachment.allInstances()->exists(a | a.id = id)
```

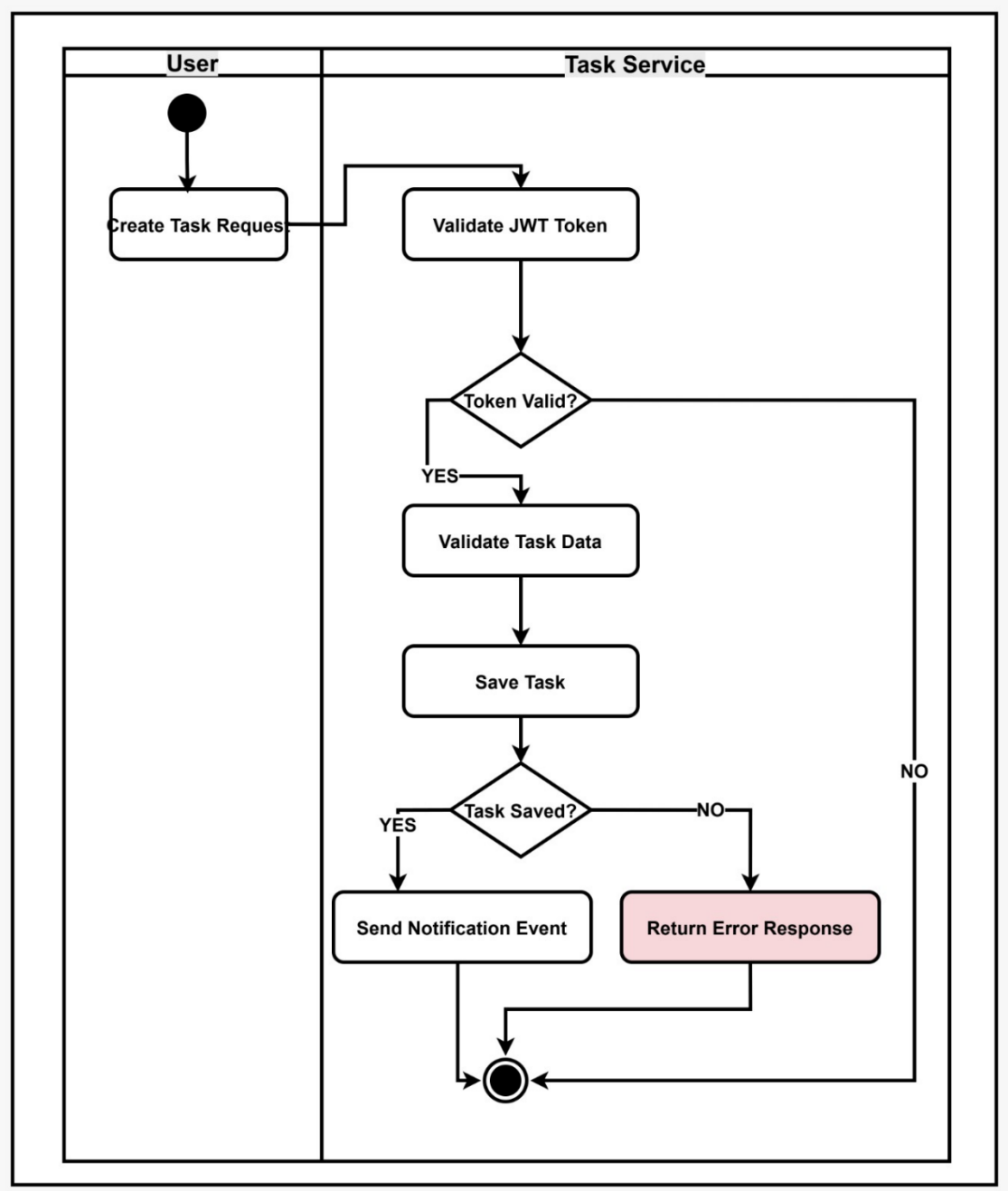
Diagrams

13-ERD (Task service)



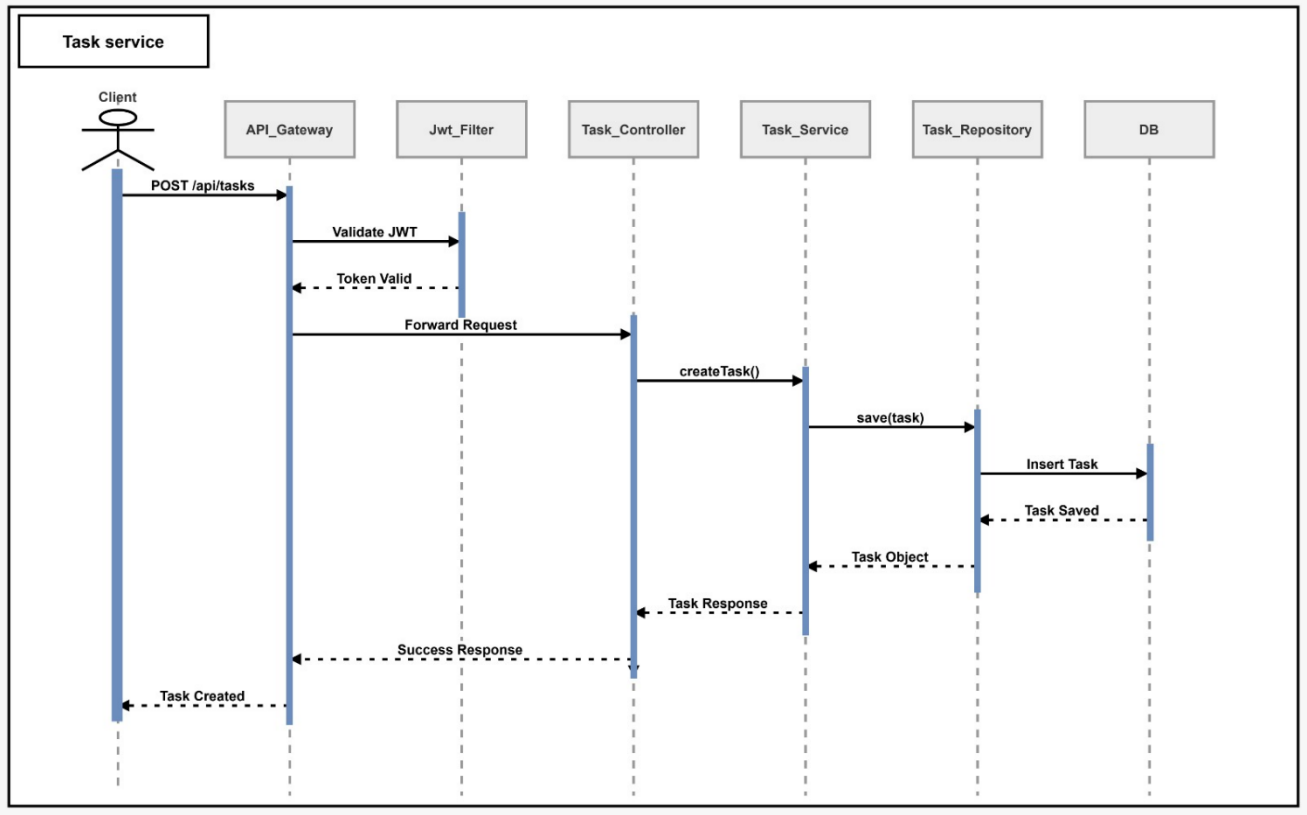
Diagrams

14-Activity Diagram (Task service)



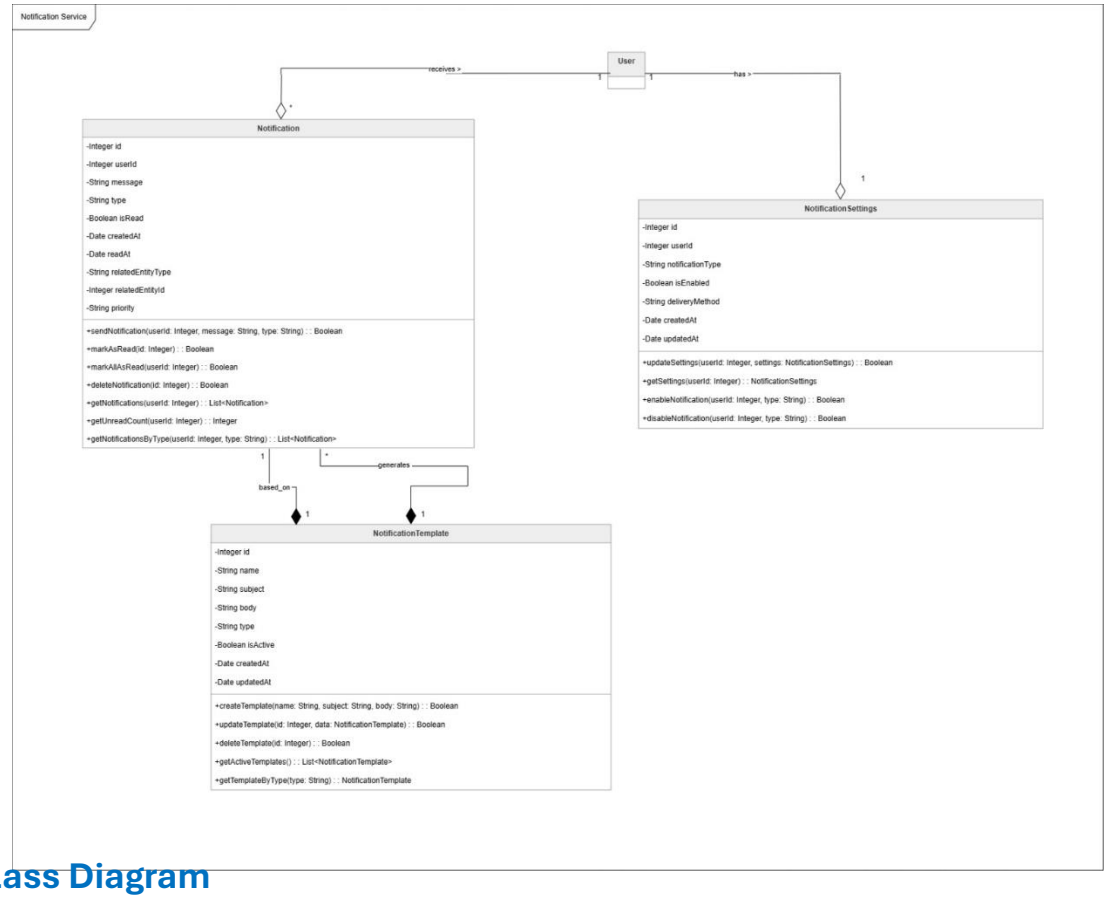
Diagrams

15-Sequence Diagram (Task service)



Diagrams

16-Class Diagram (Notification)



OCL For Class Diagram

-- Notification Class Constraints

context Notification
inv ValidMessage:
  self.message <> ''
context Notification
inv ValidType:
  self.type <> ''
context Notification
inv ValidPriority:
  self.priority = 'Low' or
  self.priority = 'Medium' or
  self.priority = 'High'
context Notification
inv ValidUserId:
  self.userId > 0
context Notification
inv ReadDateValidation:
  self.isRead = true implies self.readAt <> null
context Notification
inv CreatedBeforeRead:
  self.readAt <> null implies
  self.createdAt <= self.readAt

-- Notification Operations

context Notification::sendNotification(
  userId : Integer,
  message : String,
  type : String
) : Boolean
pre ValidInput:
  userId > 0 and
  message <> '' and

```
  type <> ''
post NotificationCreated:
  result = true implies
  Notification.allInstances()->exists(n |
    n.userId = userId and
    n.message = message and
    n.type = type
  )
context Notification::markAsRead(
  id : Integer
) : Boolean
pre NotificationExists:
  Notification.allInstances()->exists(n | n.id = id)
pre NotAlreadyRead:
  Notification.allInstances()->exists(n |
    n.id = id and n.isRead = false
  )
post NotificationMarked:
  result = true implies
  Notification.allInstances()->exists(n |
    n.id = id and
    n.isRead = true
  )
context Notification::markAllAsRead(
  userId : Integer
) : Boolean
pre UserNotificationsExist:
  Notification.allInstances()->exists(n |
    n.userId = userId and n.isRead = false
  )
post AllMarkedRead:
  result = true implies
  Notification.allInstances()->forAll(n |
    n.userId = userId implies n.isRead = true
  )
context Notification::deleteNotification(
  id : Integer
) : Boolean
pre NotificationExists:
  Notification.allInstances()->exists(n | n.id = id)
post NotificationDeleted:
  result = true implies
  Notification.allInstances()->forAll(n |
    n.id <> id
  )
context Notification::getUnreadCount(
  userId : Integer
) : Integer
post ValidCount:
  result =
  Notification.allInstances()->select(n |
    n.userId = userId and n.isRead = false
  )->size()
-- NotificationSettings Constraints
context NotificationSettings
inv ValidNotificationType:
  self.notificationType <> ''
context NotificationSettings
inv ValidDeliveryMethod:
  self.deliveryMethod = 'Email' or
```

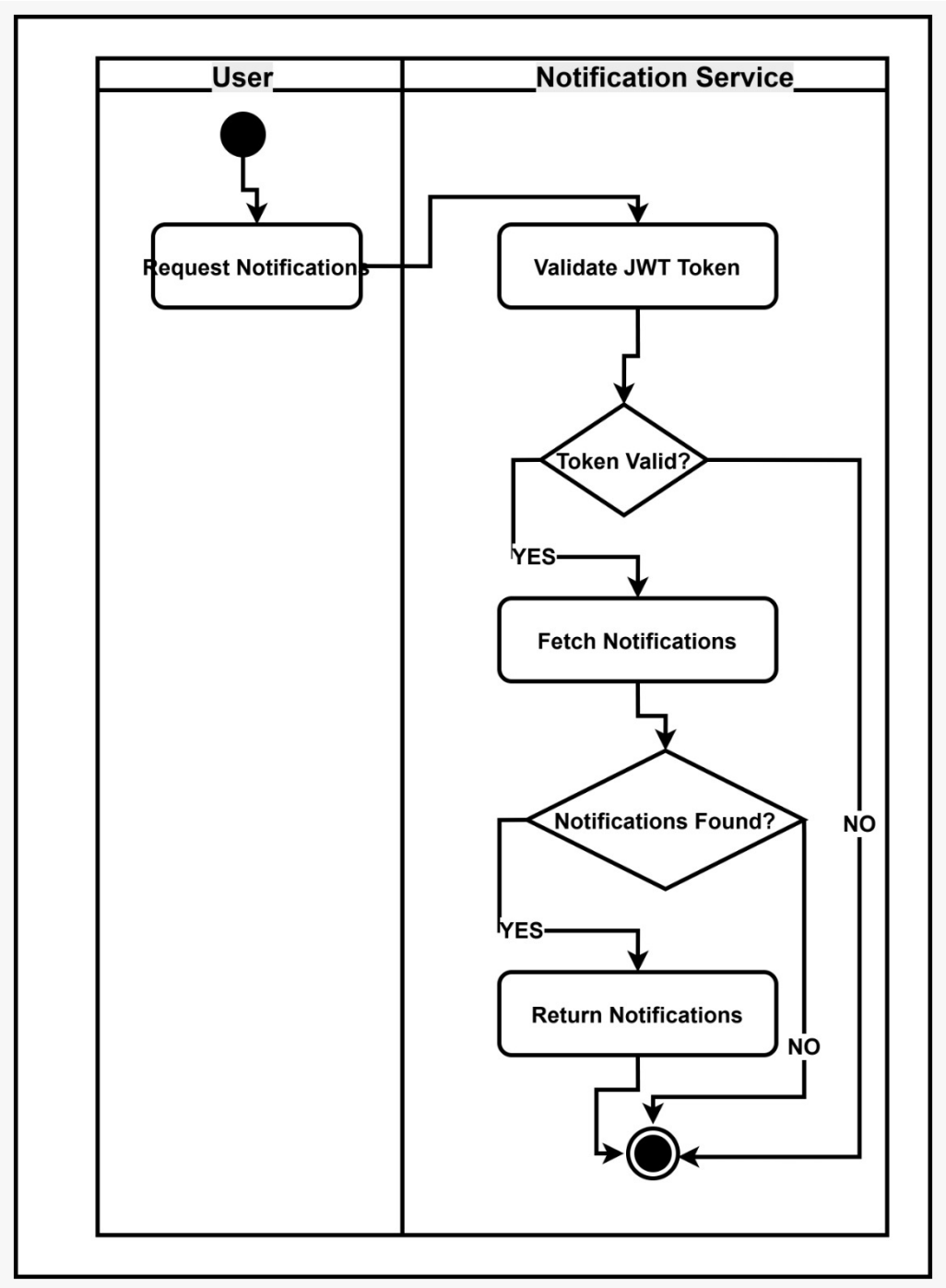
```
  self.deliveryMethod = 'SMS' or
  self.deliveryMethod = 'Push'
context NotificationSettings
inv ValidUserId:
  self.userId > 0
context NotificationSettings
inv CreatedBeforeUpdated:
  self.createdAt <= self.updatedAt
-- NotificationSettings Operations
context NotificationSettings::updateSettings(
  userId : Integer,
  settings : NotificationSettings
) : Boolean
pre SettingsExist:
  NotificationSettings.allInstances()->exists(s |
    s.userId = userId
  )
post SettingsUpdated:
  result = true implies
  NotificationSettings.allInstances()->exists(s |
    s.userId = userId and
    s.deliveryMethod = settings.deliveryMethod and
    s.isEnabled = settings.isEnabled
  )
context NotificationSettings::enableNotification(
  userId : Integer,
  type : String
) : Boolean
pre SettingsExist:
  NotificationSettings.allInstances()->exists(s |
    s.userId = userId and
    s.notificationType = type
  )
post NotificationEnabled:
  result = true implies
  NotificationSettings.allInstances()->exists(s |
    s.userId = userId and
    s.notificationType = type and
    s.isEnabled = true
  )
context NotificationSettings::disableNotification(
  userId : Integer,
  type : String
) : Boolean
pre SettingsExist:
  NotificationSettings.allInstances()->exists(s |
    s.userId = userId and
    s.notificationType = type
  )
post NotificationDisabled:
  result = true implies
  NotificationSettings.allInstances()->exists(s |
    s.userId = userId and
    s.notificationType = type and
    s.isEnabled = false
  )
-- NotificationTemplate Constraints
context NotificationTemplate
inv ValidTemplateName:
  self.name <> ''
```

```
context NotificationTemplate
inv ValidSubject:
  self.subject <> ''
context NotificationTemplate
inv ValidBody:
  self.body <> ''
context NotificationTemplate
inv ValidType:
  self.type <> ''
context NotificationTemplate
inv CreatedBeforeUpdated:
  self.createdAt <= self.updatedAt
-- NotificationTemplate Operations
context NotificationTemplate::createTemplate(
  name : String,
  subject : String,
  body : String
) : Boolean
pre ValidInput:
  name <> '' and
  subject <> '' and
  body <> ''
post TemplateCreated:
  result = true implies
  NotificationTemplate.allInstances()->exists(t |
    t.name = name and
    t.subject = subject
  )
context NotificationTemplate::updateTemplate(
  id : Integer,
  data : NotificationTemplate
) : Boolean
pre TemplateExists:
  NotificationTemplate.allInstances()->exists(t |
    t.id = id
post TemplateUpdated:
  result = true implies
  NotificationTemplate.allInstances()->exists(t |
    t.id = id and
    t.name = data.name and
    t.subject = data.subject
context NotificationTemplate::deleteTemplate(
  id : Integer
) : Boolean

pre TemplateExists:
  NotificationTemplate.allInstances()->exists(t |
    t.id = id
  )
post TemplateDeleted:
  result = true implies
  NotificationTemplate.allInstances()->forAll(t |
    t.id <> id
  )
context NotificationTemplate::getTemplateByType(
  type : String
) : NotificationTemplate
post CorrectTemplateReturned:
  result.type = type
```

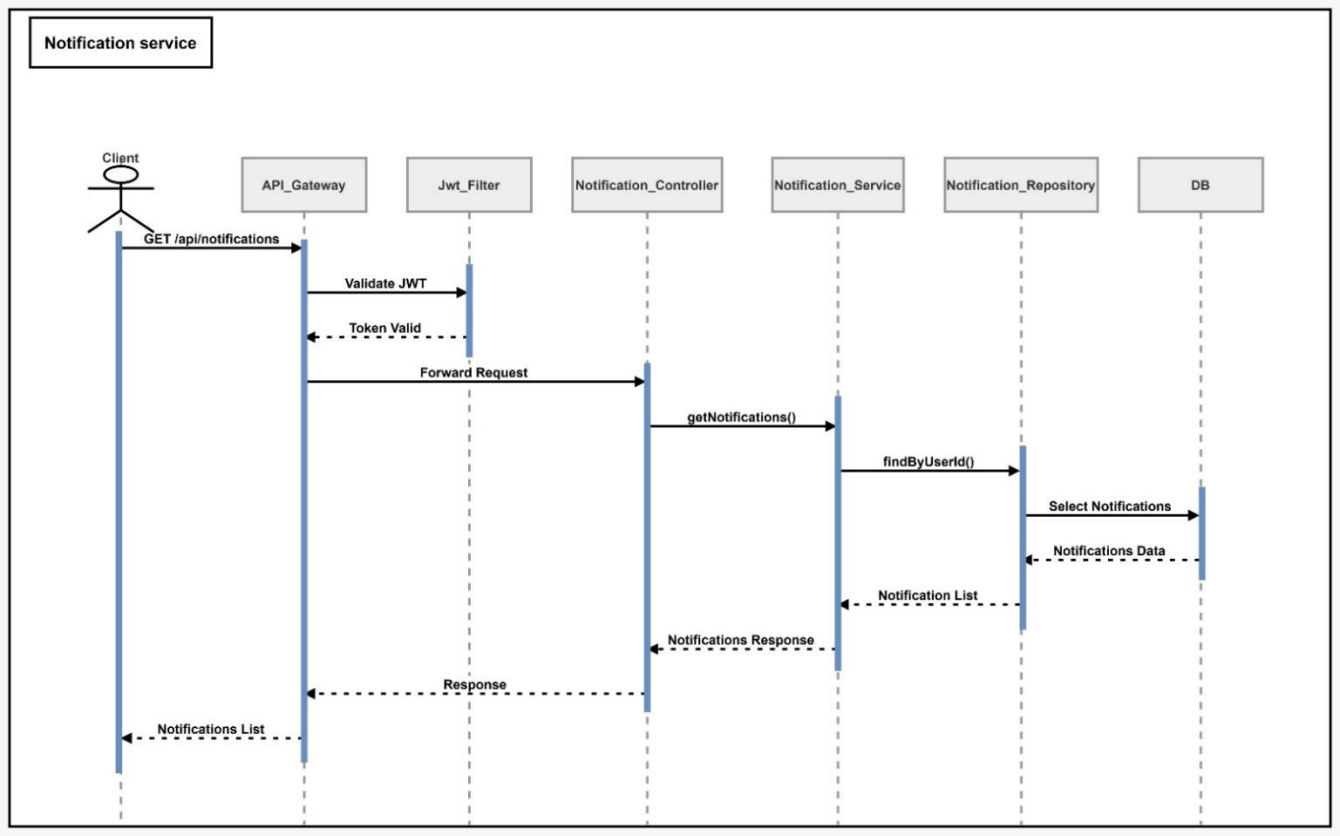
Diagrams

17-Activity Diagram (Notification)



Diagrams

18-Sequence Diagram (Notification)



TheEnd

Done By

-Kareem Mohamed: 20230419
-Kerolos Moris: 20230425
-Hussein Yasser: 20230179
-Khaled Waleed: 20230189
-Khaled Hany: 20230188
-Mohamed Yasser: 20230518